

IA en ingeniería con un esquema determinista de tres capas

Por qué el 84 % de un sistema con LLM no debería contener un LLM

Un LLM puede cumplir el contrato entero y aun así mentir

Lo que devolvió el modelo

```
puntaje_sugerido : "5.5/10" <- inventado  
nivel_desempeno : "Bueno" <- inventado  
observaciones : [ "0.5/1", "0.7/1", "0.3/1" ] <- suma 1.5
```

Obsérvese lo que *sí* fue correcto

- JSON válido
- campos presentes
- denominadores correctos
- ...y el total **no es la suma de sus propias partes.**

Un LLM no es una calculadora. Cumplir un formato no es cumplir un contrato.

La asimetría que hay que resolver

Lo que es un LLM

- Un generador de texto *plausible*
- Sin garantía de aritmética
- Sin garantía de repetibilidad
- Y sin que eso sea un defecto: es su naturaleza

Lo que exige la ingeniería

- El mismo input → el mismo output
- Que se pueda auditar *por qué*
- Que el error sea ruidoso, no silencioso

No es una discusión filosófica. Es una asimetría **medible**, y se mide en fallos reales con coste concreto.

La respuesta: separar por responsabilidad

	Capa	Artefacto	Responde	Contiene LLM
1	Directiva	directives/*.yaml	¿qué?	no
2	Orquestación	flujo/*.py, mcp_*	¿cuándo?	no (solo decide)
3	Ejecución	execution/*.py	¿cómo?	10 de 62

La regla que sostiene el patrón

La lógica de negocio no se delega al modelo. Se le pide lo único que hace bien –*producir texto*– y el resto se calcula en código.

84 %

scripts de ejecución sin ningún
LLM

10

fronteras con LLM

\$0

coste de decidir el modelo

El enrutador es una función pura

Elegir qué modelo usar *no lo decide un modelo*: lo decide código local, a partir de un descriptor medido. Mismo descriptor → mismo tier, siempre. Y no hace ni una sola llamada a la API.

Por qué esto no es un detalle de implementación

Las 10 fronteras están **declaradas**: son ficheros concretos y nombrados, no un `if` repartido por el código.

```
goal: > Evaluar el examen del estudiante enviando las páginas del PDF como imágenes  
al modelo y obteniendo una evaluación en JSON. required_inputs: - name: pdf_examen  
description: Ruta al PDF del examen. steps: - step: 1 description: Renderizar el PDF  
a 250 DPI y enviarlo en bloque al modelo con la instrucción de corrección.
```

Por qué está en YAML y no en el prompt

Porque es **configuración**, no conversación: se versiona, se revisa con un diff, y lo escribe alguien que entiende el dominio, no el modelo.

Lo que sí hace

- Leer la directiva
- Resolver entradas y decidir el orden
- Delegar cada paso en capa 3
- Validar salidas y códigos de retorno
- Registrar estado para poder reanudar

Lo que NO hace

- ...calcular
- ...sumar, comparar ni convertir
- ...procesar datos crudos
- ...decidir un modelo a ojo

Una capa de decisión delgada, con la lógica fina abajo. Por eso el mismo fallo no se propaga a veinte sitios.

- Funciones puras donde se puede: mismo input, mismo output, sin red.
- Cada script valida *sus propias* salidas y falla ruidosamente.
- Códigos de salida tipados; un 0 significa 0, no ño vi nada:
- Cobertura de pruebas como forma de documentar el contrato.

Dónde vive la verdad

Cuando hay una discrepancia, la respuesta no la da el prompt ni lo que el modelo recuerda: la da el test que falla.

Fallo 1 • El modelo no suma: el programa suma

Síntoma

Salida con status: ok, JSON perfecto, cada ítem con su denominador ...y un total que no era la suma.

Causa profunda

Se le pidió al modelo el *total*. La aritmética la hacía el LLM, y la aritmética es justo lo que un modelo no garantiza.

Arreglo

- El esquema *deja de pedir* el total
- El programa suma: función pura, testeable, sin red
- El máximo es un invariante, no una consecuencia
- Si se pasa del tope, se recorta y se *avisa*

Lo que se transfiere: en cuanto un **número importa**, el número lo calcula el programa.

El síntoma

3 ítems de 1 punto (total 3) con la mitad respondidos. Al sumar en crudo: **1.5/10** y nivel *Insuficiente*. El alumno sacaba el 50 %.

Peor: el bug espejo

Si el modelo se comía un criterio, su peso desaparecía del denominador y el resto se normalizaba **hacia arriba**: un fallo del modelo bonificaba al alumno.

El arreglo

- La escala la fija el *instrumento* (la rúbrica), no quien lo corrige
- Normalizar a la escala institucional, siempre
- Un criterio ausente vale 0 sobre esa escala: no se renormaliza
- Sin escala declarada *no se publica nota*, y se dice por qué

Fallo 3 • El modo JSON no es un esquema

```
request: response_format = {"type": "json_object"} <- solo promete un objeto  
respuesta: {"text": "...", "preguntas": []} <- objeto válido. otro esquema.
```

El agravante

`response_format` promete **una sola cosa**: que la respuesta sea un objeto. No promete las claves, ni los tipos, ni el vocabulario. Y el esquema del prompt es texto: la capa más débil del contrato.

El arreglo

La barrera real es un **validador en el programa**: vocabulario cerrado para los campos enumerados, comprobación de tipos, y la clave que decide la vialidad del flujo es *obligatoria*. Salida con código 5 y la lista de problemas; el JSON del modelo no se acepta como informe.

Cuando un número importa, el número lo calcula el programa.

Y cuando la forma importa, la forma la verifica el programa.

Aplicaciones fuera de la IA

- Metrología y conversión de unidades
- Control de calidad y criterios de aceptación
- Presupuestos y dosages
- Cualquier cosa donde un decimal de más sea un problema

Por qué así se aprende

- No es desconfiar del modelo: es asignar la aritmética a lo único que sabe hacer aritmética.

Una idea formal que se transfiere: la álgebra de veredictos

La regla

- Gana el **peor** estado, nunca el promedio
- Lo no verificado **impide** el verde
- Se distingue *no instalado* de *caído* de *parado*

Por qué importa fuera de aquí

- Es un promedio con tres estados en rojo y uno en amarillo: se ve verde mientras algo crítico no se midió
- Es exactamente la diferencia entre *sin datos* y *con buenos datos*
- Es control robusto aplicado a un pipeline de verificación

Y hay una segunda: **una sola definición**, compartida por las dos auditorías. Dos álgebras que divergen en silencio son la forma más cara de equivocarse.

Lo que el patrón *no* resuelve

- La capa 1 (directivas) aún no es verificable por auditoría: es informativa
- Un motor de asistente rotativo no es reproducible
- El hardware disponible no entrena modelos
- Sin red, parte de los proveedores no son accesibles

Cómo se presenta esto

- Un patrón cuya frontera se declara se enseña mejor que uno que se vende
- La reproducibilidad tampoco es gratis: cuesta disciplina de capa 3 y tests

1. **Escriba el SOP antes del código.** Si no sabe escribirlo, aún no sabe qué tiene que automatizar.
2. **Ponga el criterio en el programa.** Toda aritmética, toda comparación, toda conversión: en código, no en el prompt.
3. **Valide la forma, no confíe en ella.** Un JSON que se parsea no es un JSON que cumple su esquema.
4. **Defina la escala en el instrumento.** Y si no la hay, no publique el número.
5. **Verifique por registro, no por bandera verde.** Un programa que compila o termina con 0 puede estar mintiendo: lea su salida.
6. **Deje la frontera escrita.** Los ficheros con LLM se nombran; todo lo demás es código.

IA donde aporta.
Código donde se exige.
Verificación en ambos.

La arquitectura no es una jaula para el modelo:
es lo que le permite ser útil sin dejar de ser verificable.